

Gatling: A Lightweight Load Testing Tool

Gatling is the ultimate open source load testing tool for programmers. Though focused on Web applications, it can be used to analyse and measure the performance of a variety of services. As it is Scala based, it generates reports in the form of pretty graphs, from which the results can be analysed.



Gatling is a lightweight stress testing tool backed by Scala, Akka and Netty. It works asynchronously if the underlying HTTP request is non-blocking. It uses messages for virtual user simulation rather than a thread per virtual user. Because of this, it consumes comparatively less system resources to run load tests. In this article, let us look at a REST endpoint load test by taking the steps that follow.

Step 1: Download the Gatling bundle from <http://gatling.io/#/resources/download>. The folder structure of `GATLING_HOME` is explained in Figure 1.

Step 2: Set the `GATLING_HOME` and `JAVA_HOME` environment variables and write a sample simulation using Scala. Java developers do not need to have strong Scala knowledge to write Gatling simulations with Scala. A very basic knowledge is sufficient. A simulation to test a REST endpoint is given below:

```
package org.smarttechie
import io.gatling.core.Predef._
import io.gatling.http.Predef._
import scala.concurrent.duration._
class SampleRESTEndPointPrefSimulation extends Simulation {
    val httpProtocol = http.baseUrl("<base url of your end point>")
    val restEndPoint = "/posts" //provide the URI of your end point
    val restEndPointScenario = scenario("Posts_Pref_Simulation")
    .exec(http("request_1")
        .get(restEndPoint))
    setUp(restEndPointScenario.inject(rampUsers(1000)
        over(10seconds))).protocols(httpProtocol)
}
```

In this simulation, we have ramped up 1000 users in 10 seconds. Once the simulation script is ready, go to the `bin` folder and launch `gatling.bat` for a Windows system or `gatling.sh` on Linux based computers. The script will run the simulation available in the `simulations` folder and generate the reports in the `results` folder.

A sample report is given in Figure 2.

Now let's look at how to use Gatling Maven integration and additional features like feeders, externalising the properties, etc. When we integrate Gatling with Maven, it enables us to do that as part of continuous integration. Along with that, we will look at how to externalise properties used in the simulation script and the dynamic data feed using Gatling feeders.

As a first step, create a Maven project and add the Gatling dependencies `pom.xml`. You can download the `xml` file from https://github.com/2013techsmarts/Gatling_Maven_Demo.

Now, we will externalise the `baseUrl` property used in our simulation. For this, add the `application.properties` file from `src/test/resources` and use `ConfigFactory` to get access to the properties. The sample simulation is given below:

```
package org.smarttechie

import io.gatling.core.Predef._
import io.gatling.http.Predef._
import scala.concurrent.duration._
import com.typesafe.config._
class SampleRESTEndPointPrefSimulation extends Simulation {
    val conf = ConfigFactory.load()
    val baseUrl = conf.getString("baseUrl")
    val httpProtocol = http.baseUrl(baseUrl)
    val restEndPoint = "/posts"
    val restEndPointScenario = scenario("Posts_Pref_
```